

1. Introduction

1.1 Purpose

This document describes the functional and non-functional requirements of a cross-border e-commerce simulation platform for international students. It simulates purchasing products internationally through web site and a mobile application.

1.2 Scope

The system provides an online platform where students can browse products, choose their variants, read review, add to carts, apply coupons, manages shipping addresses and place orders with international shipping. The system also includes an administration interface for product management, order management, inventory control and overviewing statistics.

1.3 Definitions

Term	Meaning
SRS	Software Requirements Specification
API	Application Program Interface
SKU	Stock Keeping Unit

1.4 References

SRS STANDART: IEEE 29148

[Vue.js](#)

[Node.js](#)

[Nuxt.js](#)

[PostgreSQL](#)

[Docker](#)

[Stripe](#)

2. System Overview

2.1 Product Perspective

2.2 User Classes

1) Customer

Customers are end users who browse products and make purchases through the platform. They are registered users. **Characteristics:**

- low technical knowledge
- uses web or mobile app

- focus on usability

Main actions:

- browse products
- search/filter items
- choose different items variants
- add to cart
- apply coupons
- place orders
- track orders

2) Administrator

Administrator manages the system and have a full access to administrative features, including product management, order monitoring, managing coupons, refunds policy and system configuration.

Characteristics:

- high technical knowledge
- uses admin dashboard

Main actions:

- manage products
- manage categories
- manage coupons
- view and update orders
- manage users

3) Guest

Guest users are unregistered users who can browse product, add items to cart but may not place orders. **Main actions:**

- browse catalog
- add to catalog

2.3 Operating Environment

2.3.1 Client Environmet

Web-browsers: Chrome, Opera, Firefox, Edge, Safari

Mobile-devices: Tablets and smartphones based on Android OS

2.3.2 Server Environment

Linux-based cloud server, backend application server, relational database using PostgreSQL.

2.4 Assumptions and Dependencies

???

3. System Architecture Overview

3.1 User Registration

3.1.1 Description

The User Registration feature allows new users to create an account in the system in order to access personalized services such as placing orders, tracking purchases, managing shipping addresses, and viewing order history.

The registration process requires users to provide valid personal information including an email address and password. The system validates the provided information and creates a new user account if all validation checks are successful.

This feature is available through both the web application and the mobile application interfaces.

3.1.2 Actors

Actor	Description
User	A person who wants to create an account in the system
System	The platform responsible for validating and storing user information

3.1.3 Preconditions

The following conditions must be met before the registration process begins:

- The user has access to the web or mobile application.
- The user is not currently logged into the system.
- The system database is operational.

3.1.4 Main Flow of Events

1. The user navigates to the registration page
2. The user enters the required registration information including email address, password, and name.
3. The system validates the input data format.
4. The system verifies that the email address is not already associated with an existing account.
5. The system encrypts and stores the user password.
6. The system creates a new user account record in the database.
7. The system confirms successful registration and allows the user to log in.

3.1.5 Alternative Flows

A1: Email Already Exists

1. The system detects that the email address is already registered
2. The system displays an error message requesting the user to use another email.

A2: Invalid Data Input:

1. The system detects invalid or missing registration information.
2. The system prompts the user to correct the input fields.

3.1.6 Functional Requirements

FR-1 The system shall allow users to create a new account using an email address and password.

FR-2 The system shall validate that the provided email address is unique.

FR-3 The system shall validate the format of the email address.

FR-4 The system shall require users to provide a password meeting minimum security requirements.

FR-5 The system shall store user passwords using secure hashing algorithms.

FR-6 The system shall store user account information in the system database.

FR-7 The system shall confirm successful account creation.

3.1.7 Postconditions

After successful registration:

- A new user account is created.
- User information is stored in the database.
- The user can log into the system using their credentials

3.1.8 Data Storage

User registration data is stored in the system database. The primary table involved is the users table.

Typical stored fields include: `user_id`, `email`, `password_hash`, `name`, `registration_date`, `account_status`

3.2 User Authentication

3.2.1 Description

The User Authentication feature allows registered users to securely access their accounts by verifying their identity using login credentials. Users must provide a valid email address and password that match the credentials stored in the system.

The authentication process ensures that only authorized users can access protected resources such as order history, personal information, and account settings. After successful authentication, the system creates a session or authentication token that allows the user to interact with the system without repeatedly entering credentials.

3.2.2 Actors

Actor	Description
User	A registered user attempting to log in
System	The platform responsible for verifying user credentials

3.2.3 Preconditions

The following conditions must be met before authentication begins:

- The user must already have a registered account.
- The user must access the login interface via the web or mobile application.
- The authentication service and database must be available.

3.2.4 Main Flow of Events

1. The user navigates to the login page.
2. The user enters their email address and password.
3. The system validates the input format.
4. The system retrieves the user record associated with the email address.
5. The system compares the provided password with the stored password hash.
6. If authentication is successful, the system creates a user session or authentication token.
7. The system grants access to the user's account dashboard.

3.2.5 Alternative Flows

A1: Incorrect Credentials

1. The system detects that the password does not match the stored credentials.
2. The system displays an authentication error message.
3. The user is prompted to retry login.

A2: Non-existent Account

1. The system detects that the provided email is not associated with any account.
2. The system informs the user that the account does not exist.

A3: User forgot his password

3.2.6 Functional Requirements

FR-8 The system shall allow registered users to log in using their email address and password.

FR-9 The system shall validate the format of login credentials before processing authentication.

FR-10 The system shall verify the provided password against the stored password hash.

FR-11 The system shall create a user session or authentication token after successful authentication.

FR-12 The system shall deny access if authentication fails.

3.2.7 Postconditions

After successful authentication:

- The user is logged into the system.
- A session or authentication token is generated.
- The user gains access to protected resources such as order history and account settings.

3.2.8 Security Considerations

User authentication must follow secure security practices. Passwords must never be stored in plain text and must be protected using cryptographic hashing algorithms. All authentication requests must be transmitted through encrypted HTTPS connections to prevent interception of credentials.

3.2.9 Data Storage

Authentication relies on the following stored data: `user_id`, `email`, `password_hash`, `account_status`, `last_login_timestamp`

3.3 Product Catalog

3.3.1 Description

The Product Catalog feature provides a structured representation of all products available on the platform. It allows users to browse, view, and filter products by categories, brands, price, and other attributes. The system ensures that product information such as name, description, price, availability, images, and variants is accurate and up to date.

The catalog supports multiple product attributes (e.g., color, size), variants, and inventory levels. The system also allows administrators to manage products, categories, and associated metadata through a separate management interface.

3.3.2 Actors

Actor	Decription
User	A customer browsing or searching products
Admin	A customer browsing or searching products
System	The platform providing access and management of product information

3.3.3 Preconditions

- Users must have access to the web or mobile application.
- The system database must contain product data.
- Administrative users must be authenticated to modify the catalog.

Example database tables:

- products
- categories
- product_attributes
- product_attribute_values
- product_variants

3.3.4 Main Flow of Events

1. The user navigates to the product catalog page.

- 2. The system displays product categories or featured products.
- 3. The user selects a category or searches for a product.
- 4. The system retrieves matching product data from the database.
- 5. The system displays product details, including images, descriptions, price, availability, and variants.
- 6. The user can filter products by attributes such as color, size, brand, or price.
- 7. The system updates the displayed results according to the selected filters.

3.3.5 Alternative Flows

A1: No Products Found:

- 1. The system detects no matching products for the search/filter.
- 2. The system displays a “No products found” message.

A2: Product Out of Stock:

- 1. The system detects that a selected product variant is out of stock.
- 2. The system displays an out-of-stock indicator and disables the “Add to Cart” button.

3.3.6 Functional Requirements

- FR-19** The system shall display all active products in the catalog.
- FR-20** The system shall organize products into categories and subcategories.
- FR-21** The system shall allow filtering by product attributes (e.g., color, size, brand, price).
- FR-22** The system shall allow users to search for products using keywords.
- FR-23** The system shall display product details including name, description, images, price, stock availability, and variants.
- FR-24** The system shall allow administrative users to create, update, and delete products.
- FR-25** The system shall manage inventory for product variants and update availability accordingly.

3.3.7 Postconditions

After browsing or interacting with the product catalog:

- Users can view and filter products.
- Product information is accurately retrieved from the database.
- Inventory levels are reflected for each product variant.
- Admin changes are reflected in the catalog.

3.3.8 Data Storage

Table	Description
products	Stores main product information (name, description, price)
categories	Stores product categories and hierarchy

Table	Description
product_attributes	Defines possible attributes (size, color)
product_attribute_values	Stores possible values for attributes
product_variants	Links products to specific variants (size/color combinations)
product_images	Stores URLs or file paths for product images

3.4 Product Search & Filtering

3.4.1 Description

The Product Search & Filtering feature enables users to quickly find products in the catalog by entering keywords or applying filters. Users can search by product name, description, or SKU and can filter results by attributes such as category, brand, price range, color, size, or rating.

The system provides real-time search suggestions and ensures that only relevant, in-stock products are displayed. This feature improves user experience by helping users locate products efficiently and reduces time spent browsing.

3.4.2 Actors

Actor	Decription
User	A customer searching products
Admin	The platform responsible for executing searches, applying filters, and returning results
System	May define searchable attributes and available filters in the catalog

3.4.3 Preconditions

Users must have access to the web or mobile application. Product catalog data must exist in the database. System indexing or search services must be operational. Relevant database/search tables:

- products
- product_attributes
- product_attribute_values
- categories

3.4.4 Main Flow of Events

1. The user navigates to the search bar or product listing page.
2. The user enters a search keyword or selects a filter.
3. The system validates the input.
4. The system queries the product catalog and applies filters.
5. The system returns a list of products matching the search and filter criteria.
6. The user can sort results by relevance, price, popularity, or rating.
7. The system updates displayed results in real time as filters or search terms are modified.

3.4.5 Alternative Flows

A1: No Matching Products:

- 1. The system detects that no products match the search or filter criteria.
- 2. The system displays a “No products found” message with suggestions to broaden the search.

A2: Out-of-Stock Filtering

- 1. The system detects product variants that are out of stock.
- 2. The system removes or marks out-of-stock products from search results if user selects “In Stock Only” filter.

3.4.6 Functional Requirements

- FR-26** The system shall allow users to search products by keyword (name, description, SKU).
- FR-27** The system shall allow users to filter products by attributes such as category, brand, price, size, and color.
- FR-28** The system shall return only products that are in stock unless the user selects otherwise.
- FR-29** The system shall allow users to sort search results by relevance, price, popularity, or rating.
- FR-30** The system shall provide real-time search suggestions based on partial input.
- FR-31** The system shall handle large product catalogs efficiently and return results within 2 seconds.
- FR-32** The system shall allow administrators to define searchable attributes and available filters.

3.4.7 Postconditions

After performing a search or applying filters:

- Users can view a list of products that match search and filter criteria.
- Only relevant and in-stock products are displayed.
- Users can sort or refine results as needed.

3.4.8 Data Storage

The feature interacts with:

Table	ServiceDescription
products	Stores product information (name, description, price, SKU)
product_attributes	Defines attributes like color, size, brand
product_attribute_values	Contains actual values for attributes
categories	Stores category hierarchy
search_index	Optional search engine index for fast querying

3.5 Shopping Cart

3.5.1 Description

The Shopping Cart feature allows users to temporarily store selected products before making a purchase. Users can add, update, or remove products, choose product variants, and see the total cost including taxes and shipping estimates.

The system ensures that product availability and pricing are updated in real time. The shopping cart also integrates with the Coupon System, enabling users to apply discounts automatically.

3.5.2 Actors

Actors	Description
User	A customer selecting products for purchase
System	The platform responsible for managing cart contents and calculating totals
Admin	May configure cart-related rules, e.g., maximum items, stock limits

3.5.3 Preconditions

The user has access to the web or mobile application. Products exist in the catalog and have available stock. The shopping cart service and database are operational.

Relevant database tables:

- carts
- cart_items
- products
- product_variants
- coupons

3.5.4 Main Flow of Events

1. The user adds a product to the shopping cart.
2. The system checks product availability in inventory.
3. The system adds the selected product and variant to the cart.
4. The user can update quantities or remove items.
5. The system recalculates the total cost, taxes, and shipping estimates.
6. The user can apply a coupon code, and the system calculates the discounted total.
7. The user proceeds to checkout.

3.5.5 Alternative Flows

A1: Product Out of Stock

1. The system detects that the requested quantity exceeds available stock.
2. The system displays a message and adjusts the quantity to the maximum available.

A2: Coupon Invalid or Expired

1. The user applies a coupon code.
2. The system validates the coupon.
3. If the coupon is invalid or expired, the system displays an error and does not apply the discount.

A3: Guest User

1. A non-logged-in user adds items to the cart.
2. The system stores the cart temporarily in the session or local storage.
3. If the user logs in, the system merges the guest cart with the user’s persistent cart.

3.5.6 Functional Requirements

- FR-33** The system shall allow users to add products to the shopping cart.
- FR-34** The system shall validate product availability before adding to the cart.
- FR-35** The system shall allow users to update quantities or remove items from the cart.
- FR-36** The system shall calculate the total cost including taxes, shipping estimates, and applied discounts.
- FR-37** The system shall allow users to apply valid coupon codes to the cart.
- FR-38** The system shall store shopping cart data persistently for logged-in users.
- FR-39** The system shall temporarily store shopping cart data for guest users during the session.
- FR-40** The system shall prevent checkout if items are out of stock.

3.5.7 Postconditions

After interacting with the shopping cart:

- The cart contains the selected products with updated quantities.
- The total price reflects any taxes, shipping estimates, and applied coupons.
- Stock availability is updated when items are reserved for checkout.
- Guest cart data is maintained temporarily; logged-in users’ carts are stored in the database.

3.5.8 Data Storage

The shopping cart feature interacts with these tables:

Table	Description
carts	Stores shopping cart metadata (user_id, cart_id, timestamps)
cart_items	Stores individual products and variants added to the cart
products	References product details
product_variants	References product variant details (size, color)
inventory	Checks stock levels
coupons	Stores available coupon codes and rules

3.6 Coupon System

3.6.1 Description

The Coupon System allows users to apply discount codes to their shopping cart or specific products during checkout. Coupons can provide fixed amount discounts, percentage-based discounts, free shipping, or promotional offers.

The system automatically validates coupons for eligibility, expiration, and usage limits. Discounts are calculated and applied to the shopping cart in real time, and the final discounted total is reflected in the order summary. The system also tracks coupon usage history for reporting and analytics.

3.6.2 Actors

Actor	Description
User	A customer applying a coupon during checkout
System	The platform responsible for validating coupons and applying discounts
Admin	A system administrator creating and managing coupons

3.6.3 Preconditions

- The user must have items in the shopping cart.
- The user must have access to the web or mobile application.
- Coupons must exist in the database and have defined rules. Relevant database tables:
 - coupons
 - coupon_usage
 - carts
 - cart_items
 - products

3.6.4 Main Flow of Events

1. The user enters a coupon code in the shopping cart or checkout page.
2. The system validates the coupon code for:
 - Validity and expiration date
 - Minimum order value (if applicable)
 - Product or category restrictions
 - User eligibility (e.g., new customer only)
 - Usage limits per user or overall
3. If the coupon is valid, the system calculates the discount.
4. The system applies the discount to the cart total in real time.
5. The user sees the updated total with the applied discount.
6. The coupon usage is logged in the system for future reference.

3.6.5 Alternative Flows

A1: Invalid Coupon

- 1. The user enters a coupon code.
- 2. The system detects that the code is invalid, expired, or not applicable.
- 3. The system displays an error message explaining the reason.
- 4. No discount is applied to the cart.

A2: Coupon Usage Limit Reached

- 1. The user applies a coupon code that has reached its maximum usage.
- 2. The system denies the coupon application and displays a message.

A3: Coupon Not Eligible for Selected Products

- 1. The system detects that the coupon is not valid for products in the cart.
- 2. The system displays a message and does not apply the discount.

3.6.6 Functional Requirements

- FR-41** The system shall allow users to enter coupon codes in the shopping cart or checkout page.
- FR-42** The system shall validate coupons for expiration, eligibility, and usage limits.
- FR-43** The system shall apply valid coupons to the shopping cart total automatically.
- FR-44** The system shall calculate and display the discounted total in real time.
- FR-45** The system shall track coupon usage per user and overall.
- FR-46** The system shall allow administrators to create, edit, or deactivate coupons.
- FR-47** The system shall allow different types of discounts, including fixed amount, percentage-based, or free shipping.
- FR-48** The system shall prevent multiple incompatible coupons from being applied simultaneously if defined in business rules.

3.6.7 Postconditions

After applying a coupon:

- The shopping cart total is updated to reflect the discount.
- Coupon usage is recorded in the system.
- The user can proceed to checkout with the discounted total.

3.6.8 Data Storage

The Coupon System interacts with the following tables:

Table	Description
coupons	Stores coupon codes, types, discount rules, expiration dates, and usage limits
coupon_usage	Tracks individual and overall coupon usage
carts	References cart totals and applied discounts

Table	Description
cart_items	References products to which coupons may apply
products	Optional table for product-specific coupon restrictions

3.7 Order Processing

Description

The Order Processing feature allows users to complete purchases by converting their shopping cart into an order. The system validates product availability, applies any discounts or coupons, calculates taxes and shipping costs, and generates a unique order record.

Once an order is created, the system updates inventory levels, records payment information, and triggers notifications to both the user and the administration. Orders are stored immutably to maintain accurate historical records for tracking, analytics, and customer service.

3.7.2 Actors

Actor	Description
User	A customer completing a purchase
System	The platform responsible for validating, processing, and storing orders
Payment Gateway	External service that processes payment transactions
Admin	Monitors orders, updates status, and manages exceptions

3.7.3 Preconditions

- The user must have at least one item in their shopping cart.
- The user must be authenticated (if required by business rules).
- Inventory and payment services must be operational.
- Any applied coupons must be validated. Relevant database tables:
 - orders
 - order_items
 - carts
 - cart_items
 - inventory
 - coupons
 - payments

3.7.4 Main Flow of Events

1. The user reviews the shopping cart and proceeds to checkout.
2. The system validates product availability for each item in the cart.
3. The system calculates final totals, including discounts, taxes, and shipping.
4. The system collects payment details and submits the transaction to the payment gateway.

5. Upon successful payment, the system creates a new order record in the orders table and order items in the order_items table.
6. Inventory levels are updated to reflect reserved or sold quantities.
7. Notifications (email or in-app) are sent to the user confirming the order.
8. The order status is initialized as "Pending" or "Processing" and can be updated by the admin as fulfillment progresses.

3.7.5 Alternative Flows

A1: Payment Failure

1. The payment gateway declines the transaction.
2. The system displays an error message and allows the user to retry or choose another payment method.
3. The order is not created and inventory is not updated.

A2: Insufficient Stock During Checkout

1. The system detects that one or more items in the cart are out of stock.
2. The system informs the user which items cannot be fulfilled and allows them to adjust quantities or remove items.

A3: Coupon Invalid at Checkout

1. A previously applied coupon becomes invalid before order confirmation (e.g., usage limit reached).
2. The system removes the discount, recalculates totals, and notifies the user.

3.7.6 Functional Requirements

FR-49 The system shall convert a shopping cart into an order upon checkout.

FR-50 The system shall validate product availability before creating an order.

FR-51 The system shall calculate final order totals including taxes, shipping, and discounts.

FR-52 The system shall integrate with external payment gateways to process payments.

FR-53 The system shall create order records and associated order items in the database.

FR-54 The system shall update inventory levels immediately after successful order creation.

FR-55 The system shall send order confirmation notifications to the user.

FR-56 The system shall maintain immutable historical records of all orders for auditing and analytics.

FR-57 The system shall allow administrators to update order status (Pending, Processing, Shipped, Delivered, Cancelled).

FR-58 The system shall handle failures in payment or stock validation gracefully and notify the user.

3.7.7 Postconditions

After successful order processing:

- An order is created and stored in the database.
- Order items are associated with the order.
- Inventory is updated to reflect sold items.
- Payment is successfully recorded.
- Users receive confirmation notifications.
- Order status can be updated during fulfillment.

3.7.8 Data Storage

Table	Description
orders	Stores order-level information (order_id, user_id, totals, status, timestamps)
order_items	Stores details of each item in the order (product_id, variant, quantity, price)
carts	References the original cart used to create the order
cart_items	References items in the cart
inventory	Updates stock levels after purchase
coupons	Applies any discounts used in the order
payments	Records payment transactions and statuses

3.8 Payment Process

3.8.1 Description

The Payment Processing feature enables users to pay for their orders securely using various payment methods, including credit/debit cards, digital wallets, bank transfers, and international payment gateways.

The system validates payment details, communicates with external payment providers, and ensures that transactions are authorized and recorded. Successful payments trigger order confirmation and inventory updates, while failed payments are reported to the user with clear error messages. The system also supports multi-currency processing for cross-border purchases.

3.8.2 Actors

Actor	Description
User	A customer making a payment for their order
System	The platform responsible for processing payments securely and updating order status
Payment Gateway	External service that authorizes and processes transactions
Admin	Monitors transactions, resolves payment failures, and manages payment methods

3.8.3 Preconditions

- The user has created an order in the system.
- Payment gateways and network connections are operational.

- The user has valid payment credentials.
- The shopping cart and coupon discounts have been finalized. Relevant database tables:
- orders
- payments
- payment_methods
- currencies

3.8.4 Main Flow of Events

1. The user selects a payment method during checkout.
2. The system validates the provided payment details (card number, CVV, expiry, or wallet credentials).
3. The system calculates the total payment amount, including discounts, taxes, and shipping.
4. The system submits the payment request to the external payment gateway.
5. The payment gateway returns the transaction status (success, failure, or pending).
6. If the payment is successful:
 - The system updates the payment record in the database.
 - The associated order status is updated to "Paid" or "Processing."
 - Inventory is adjusted for purchased products.
 - Confirmation notifications are sent to the user.
7. If the payment fails, the system informs the user and allows retry or an alternative payment method.

3.8.5 Alternative Flows

A1: Payment Failure

1. The payment gateway declines the transaction.
2. The system displays an error message specifying the failure reason.
3. The user can retry payment or select another payment method.

A2: Currency Conversion Error (Cross-Border Payments)

1. The system attempts to convert the order total to the user's currency.
2. If conversion fails, the system notifies the user and requests confirmation or retry.

A3: Partial Payment / Pending Payment

1. The payment gateway returns a pending status.
2. The system marks the payment as "Pending" and restricts order fulfillment until confirmation.

3.8.6 Functional Requirements

FR-59 The system shall allow users to select a payment method during checkout.

FR-60 The system shall validate payment details before submitting transactions.

FR-61 The system shall calculate the total payment amount including taxes, shipping, and discounts.

FR-62 The system shall securely transmit payment requests to external payment gateways.

FR-63 The system shall record the status of each payment in the database.

- FR-64** The system shall update the order status to “Paid” upon successful payment.
- FR-65** The system shall send payment confirmation notifications to the user.
- FR-66** The system shall handle failed or pending payments gracefully and allow retries.
- FR-67** The system shall support multiple currencies for cross-border payments.
- FR-68** The system shall allow administrators to configure available payment methods and monitor transactions.

3.8.7 Postconditions

After payment processing:

- The order is marked as paid or pending depending on the transaction outcome.
- Payment records are stored securely in the database.
- Inventory is updated for successfully paid orders.
- Users receive notifications of payment status.

3.8.8 Data Storage

Table	Description
payments	Stores payment records including amount, method, currency, status, and timestamps
payment_methods	Stores available payment methods and configurations
orders	References the related order for which payment was made
currencies	Stores exchange rates for cross-border transactions

3.9 Order Tracking & Shipment

3.9.1 Description

The Order Tracking & Shipment feature allows users to monitor the status of their orders from placement to delivery. Users can view real-time updates on order processing, shipping, and expected delivery times.

The system integrates with logistics and courier services to track shipments and provides notifications at key stages, such as order confirmation, shipment dispatched, in transit, out for delivery, and delivered. Admins can manage shipment details, update statuses, and resolve delivery exceptions.

3.9.2 Actors

Actor	Description
User	A customer tracking the progress of their order
System	The platform providing real-time order status and shipment tracking
Courier / Logistics Partner	External service providing shipment updates and tracking numbers
Admin	Manages shipment records, updates statuses, and resolves issues

3.9.3 Preconditions

- The user must have successfully placed an order.
- The order must have a valid shipping address.
- Logistics partners or courier tracking APIs must be operational. Relevant database tables:
 - orders
 - shipments
 - shipment_status_history
 - users
 - addresses

3.9.4 Main Flow of Events

1. The user navigates to the order tracking page.
2. The system retrieves the order and shipment information from the database.
3. The system displays the current status of the order (e.g., Processing, Shipped, In Transit, Out for Delivery, Delivered).
4. The system integrates with courier APIs to update shipment status automatically.
5. The user receives notifications for important events (e.g., shipped, out for delivery, delivered).
6. Admins can manually update shipment status if needed and resolve exceptions.

3.9.5 Alternative Flows

A1: Courier API Failure

1. The system cannot retrieve real-time shipment data due to API failure.
2. The system displays the last known shipment status and notifies the user of a temporary delay in tracking updates.

A2: Address or Delivery Exception

1. The courier reports an address or delivery problem.
2. The system notifies the user and admin to resolve the issue.

3.9.6 Functional Requirements

FR-69 The system shall allow users to view the current status of their orders.

FR-70 The system shall update order status in real time as it progresses through processing and shipment.

FR-71 The system shall integrate with courier or logistics partner APIs for tracking information.

FR-72 The system shall maintain a history of shipment status updates.

FR-73 The system shall send notifications to users at key milestones (e.g., shipped, out for delivery, delivered).

FR-74 The system shall allow administrators to update shipment status manually in case of exceptions.

FR-75 The system shall validate shipping addresses before dispatch to prevent errors.

FR-76 The system shall allow tracking multiple orders simultaneously for the same user.

3.9.7 Postconditions

After tracking or updating shipments:

- sers can view real-time order and shipment status.
- Shipment history is recorded in the database.
- Notifications are sent for key updates.
- Admins can manage and correct shipment records if exceptions occur.

3.9.8 Data Storage

Table	Description
shipments	Stores shipment information including tracking numbers, courier, shipping method, and estimated delivery
shipment_status_history	Records all status updates for each shipment
orders	Links shipments to the corresponding orders
addresses	Stores shipping addresses for orders
users	References users for tracking and notifications

4. External Interface Requirements

4.1 User Interface

UI prototypes

4.2 API Interface

Below you can find a list of core API endpoints for communication between frontend and backend.

Endpoint	Method	Description	Request Parameters	Response	Error Codes	Auth
/auth/register	POST	Register a new user account	name, email, password	User object	400, 409	No
/auth/login	POST	Authenticate user and return JWT token	email, password	JWT token	400, 401	No
/users	GET	Retrieve all users (admin only)	page, limit, search (by name/email)	JSON array	401, 403, 500	Admin

Endpoint	Method	Description	Request Parameters	Response	Error Codes	Auth
/users/{id}	GET	Retrieve user information	id	User object	401, 404	Yes
/products	GET	Retrieve product catalog	category, search, min_price, max_price, sort, page, limit	JSON array	400, 500	No
/products/{id}	GET	Retrieve product details	id	Product object	404	No
/cart	GET	Retrieve current user's cart	JWT token	Cart object	401	Yes
/cart/items	POST	Add product to cart	product_id, quantity	Cart object	400, 401	Yes
/orders	POST	Create a new order	cart_items, shipping_address, payment_method	Order object	400, 401, 409	Yes
/orders/{id}	GET	Retrieve order details	id	Order object	401, 404	Yes
/users/{id}/orders	GET	Retrieve orders for a specific user	status, date_from, date_to, page, limit	JSON array	401, 404	Yes
/reviews	POST	Create a product review	product_id, rating, comment	Review object	400, 401	Yes
/products/{id}/reviews	GET	Retrieve reviews for product	rating, page, limit	JSON array	404	No
/coupons/{code}	GET	Retrieve coupon information	code	Coupon object	404	No
/admin/users	GET	Retrieve all users (admin)	page, limit, search (name/email)	JSON array	401, 403	Admin
/admin/products	POST	Create new product (admin)	name, price, category, stock	Product object	400, 401, 403	Admin

Endpoint	Method	Description	Request Parameters	Response	Error Codes	Auth
/admin/products/{id}	PUT	Update product information (admin)	id, product data	Product object	400, 401, 403	Admin
/admin/products/{id}	DELETE	Remove product (admin)	id	Success message	401, 403, 404	Admin
/admin/orders	GET	Retrieve all orders (admin)	status, date_from, date_to, page, limit	JSON array	401, 403	Admin
/admin/orders/{id}	PUT	Update order status (admin)	id, status	Order object	400, 401, 403	Admin
/admin/reviews/{id}	DELETE	Remove inappropriate review (admin)	id	Success message	401, 403, 404	Admin

Notes on Filters and Pagination Products: **category** — filter by category ID **search** — search by name/description **min_price** / **max_price** — price range **sort** — e.g., price_asc, price_desc, newest **page** / **limit** — pagination

Orders: **status** — filter by order status (pending, shipped, completed, canceled) **date_from** / **date_to** — filter by order date **page** / **limit** — pagination

Users (admin only): **search** — by name or email **page** / **limit** — pagination

Reviews: **rating** — filter by star rating **page** / **limit** — pagination

5. Non-Functional Requirements

5.1 Performance

The system shall provide responsive and efficient performance for users accessing the platform through web and mobile applications.

The system shall respond to standard API requests within 2 seconds under normal operating conditions. Product catalog pages shall load within 3 seconds for typical network connections. The system shall support simultaneous access by multiple users without significant degradation in performance. Database queries related to product search and order processing shall be optimized to ensure fast response times. The system shall implement pagination and filtering mechanisms to reduce large data transfers. Content such as product images shall be delivered using a Content Delivery Network (CDN) to improve performance for international users.

5.2 Security

The system shall ensure secure handling of user data, payment information, and communication between clients and servers.

All communication between clients and the server shall be conducted using HTTPS encryption. User authentication shall be implemented using JSON Web Tokens (JWT). Sensitive information such as passwords shall be stored using secure hashing algorithms. The system shall implement role-based access control, restricting administrative operations to authorized administrators. Payment transactions shall be processed through a trusted external payment gateway to ensure compliance with payment security standards. The system shall validate all user input to prevent common security threats such as SQL injection and cross-site scripting (XSS).

5.3 Reliability

The system shall provide stable and consistent service availability to users.

The platform shall maintain high system availability (at least 99% uptime). The system shall implement error handling mechanisms to manage unexpected failures. Database backups shall be performed regularly to prevent data loss. In case of system failure, the system shall be able to recover essential services without data corruption. Order processing and payment confirmation processes shall ensure transaction integrity to prevent duplicate or incomplete orders.

5.4 Scalability

The system shall be capable of handling increasing numbers of users and transactions as the business grows.

The system architecture shall support horizontal scaling of backend services. The database design shall allow efficient handling of large product catalogs and order histories. The system shall support increased traffic during seasonal sales and promotional events without significant performance degradation. Static content such as images shall be served through scalable storage and delivery systems. The API shall support efficient data retrieval using pagination and filtering to handle large datasets.

6. System-models

Diagrams

7. Appendices
